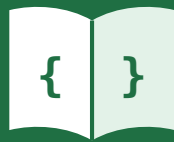

Module 06 — Spring Data JPA and Hibernate

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 JPA, Hibernate and Spring Data JPA	2
2.2 The Persistence Context and the Entity Lifecycle	2
2.3 Who Owns the Schema?	3
3. Step-by-Step Examples	3
3.1 Entities and Relationships	3
3.2 Derived Queries, Paging and Sorting	5
3.3 The N+1 Problem	6
3.4 Projections	7
3.5 Dynamic Search: Specification	7
3.6 Auditing and Optimistic Locking (@Version)	9
3.7 Pessimistic Locking	9
4. Common Mistakes and Best Practices	10
5. Summary	11
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Map JPA entities and their relationships (1-N, N-N) correctly
- Use derived queries, paging and sorting with Spring Data JPA
- Detect the N+1 query problem by measuring it, and fix it with `@EntityGraph` or `join fetch`
- Read only the data you need with interface and record projections
- Write dynamic searches with `Specification`
- Fill auditing fields automatically
- Tell optimistic (`@Version`) from pessimistic (`SELECT ... FOR UPDATE`) locking

Prerequisites: Module 05 (Flyway, transactions, Testcontainers) · **Estimated time:** 6 hours · **Docker required**

2. Concepts

2.1 JPA, Hibernate and Spring Data JPA

- **JPA (Jakarta Persistence):** the standard API for mapping objects to tables (`@Entity`, `EntityManager`).
- **Hibernate 7:** the JPA implementation Spring Boot uses. It generates the SQL.
- **Spring Data JPA:** generates queries from repository interfaces, and adds paging and specification support.

2.2 The Persistence Context and the Entity Lifecycle

Within a transaction, Hibernate keeps every entity it loads in a **persistence context** (the first-level cache):

- Asking for the same id again runs no SQL and returns the same object.
- **Dirty checking:** changing a managed entity is enough. At commit Hibernate finds the difference and writes an `UPDATE`. No `save()` call needed.
- **Lazy loading:** relationships (collections by default) are loaded only when accessed, inside an open transaction.

State	Meaning
Transient	Created with new, Hibernate does not know it
Managed	In the persistence context, changes are tracked

State	Meaning
Detached	The transaction is over, no longer tracked
Removed	Marked for deletion

2.3 Who Owns the Schema?

In this course **Flyway** manages the schema. Hibernate only validates that the entities match it (ddl-auto: validate). Never use ddl-auto: update in production: its changes cannot be reviewed or rolled back.

3. Step-by-Step Examples

With Docker running, start the application:

```
./mvnw -pl modules/06-data-jpa-postgres/lesson -am spring-boot:run
```

The JPA settings:

```
jpa:
  open-in-view: false           # no lazy loading in the web layer: queries stay in the
  ↪ service layer
  hibernate:
    ddl-auto: validate          # Flyway owns the schema; Hibernate only checks that
  ↪ entities match it
  properties:
    hibernate:
      generate_statistics: true # Lesson 3.3: count SQL statements (keep it off in
  ↪ production)
```

3.1 Entities and Relationships

Goal: map author 1-N book and book N-N category.

```
@Entity
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // bigserial in PostgreSQL
    private @Nullable Long id;
```

```

private String name;

@OneToMany(mappedBy = "author")           // Book.author owns the
    ↪ foreign key
private List<Book> books = new ArrayList<>(); // collections are LAZY by
    ↪ default

protected Author() {                     // required by JPA
    this.name = "";
}

```

```

@Entity
@EntityListeners(AuditingEntityListener.class) // fills @CreateDate /
    ↪ @LastModifiedDate
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private @Nullable Long id;

    private String isbn;
    private String title;
    private BigDecimal price;
    private int stock;

    @ManyToOne(fetch = FetchType.LAZY, optional = false) // default for @ManyToOne is
        ↪ EAGER – avoid it
    @JoinColumn(name = "author_id")
    private Author author;

    @ManyToMany
    @JoinTable(name = "book_category",
        joinColumns = @JoinColumn(name = "book_id"),
        inverseJoinColumns = @JoinColumn(name = "category_id"))
    private Set<Category> categories = new HashSet<>();

    @CreateDate
    private @Nullable Instant createdAt;

    @LastModifiedDate
    private @Nullable Instant updatedAt;

    @Version // optimistic locking
    private @Nullable Long version;
}

```

Entities are mutable classes, not records. They change their state through meaningful

methods rather than setters:

```
public void changePrice(BigDecimal newPrice) {           // behaviour instead of setters
    if (newPrice.signum() < 0) {
        throw new IllegalArgumentException("Price must not be negative");
    }
    this.price = newPrice;
}

public void removeStock(int quantity) {
    if (quantity > stock) {
        throw new IllegalStateException("Only " + stock + " left");
    }
    this.stock -= quantity;                               // no save() needed: dirty
    ↪ checking at commit
}
```

Expected output:

```
== 3.1 Entities and relationships
Effective Java by Joshua Bloch, categories [best-practices, java]
```

Its test: catalog/BookRepositoryTest.mapsTheRelationships

! Warning

@ManyToOne and @OneToOne are **EAGER** by default: every book read also reads its author. Always write `fetch = FetchType.LAZY`, and load related data together deliberately when you need it (section 3.3).

3.2 Derived Queries, Paging and Sorting

Goal: generate a query from a method name and get the results page by page.

```
Optional<Book> findByIsbn(String isbn);

Page<Book> findByTitleContainingIgnoreCase(String text, Pageable pageable); // paging +
↪ sorting for free
```

Called with `PageRequest.of(0, 2, Sort.by("price").descending())`, Spring Data runs two queries: the page data and the total count.

Expected output:

== 3.2 Derived query with paging

[Modern Java in Action, Effective Java] – page 1 of 2, 3 matches

3.3 The N+1 Problem

Goal: measure and fix the most common performance killer.

Reading all authors and touching each author's books runs **1 + N** queries: one for the authors, then a separate query per author. The fix is to load the needed relationship in the same query:

```
public interface AuthorRepository extends JpaRepository<Author, Long> {

    Optional<Author> findByName(String name);

    // findAll() (inherited): 1 query for authors, then 1 query per author when books are
    ↪ touched → N+1

    @EntityGraph(attributePaths = "books")                // fix 1: fetch the books in
    ↪ the same query
    List<Author> findAllWithBooksBy();

    @Query("select distinct a from Author a left join fetch a.books") // fix 2: JPQL
    ↪ join fetch
    List<Author> findAllFetchingBooks();
}
```

The test counts the executed SQL statements with Hibernate statistics:

```
@Test
void lazyCollectionsCauseOneQueryPerAuthor() {
    List<Author> all = authors.findAll();
    int books = countBooks(all);

    assertThat(books).isEqualTo(6);
    assertThat(statistics.getPrepareStatementCount()).isEqualTo(1 + all.size()); // 1 for
    ↪ authors + 1 per author
}
```

Expected output:

== 3.3 N+1

findAll + touching books: 6 SQL statements

@EntityGraph: 1 SQL statement

join fetch: 1 SQL statement

★ Tip

While developing, look at the generated SQL with logging.level.org.hibernate.SQL: debug. If the same select appears again and again while you loop over a list, that is an N+1.

3.4 Projections

Goal: read only the columns you need, not the whole entity.

```
List<BookTitleAndPrice> findByAuthorNameOrderByTitle(String authorName);    //  
↪ interface projection  
  
@Query("""  
    select new com.springbootedu.datajppapostgres.catalog.BookCard(b.title, a.name,  
↪    b.price)  
    from Book b join b.author a  
    where b.price < :max  
    order by b.price""")  
List<BookCard> findCardsCheaperThan(BigDecimal max);    // record  
↪ (DTO) projection
```

Projections are not managed entities: no dirty checking, no lazy loading. That makes them faster and safer for lists and reports.

Expected output:

```
== 3.4 Projections  
interface: Effective Java 89.90  
interface: Java Puzzlers 55.00  
record:    BookCard[title=Java Puzzlers, author=Joshua Bloch, price=55.00]  
record:    BookCard[title=Refactoring, author=Martin Fowler, price=85.00]
```

3.5 Dynamic Search: Specification

Goal: build one flexible query from however many criteria the user fills in.

Small, reusable query building blocks:

```
final class BookSpecifications {  
  
    private BookSpecifications() {  
    }  
}
```

```

static Specification<Book> titleContains(String text) {
    return (book, query, cb) -> cb.like(cb.lower(book.get("title")), "%" +
        ↪ text.toLowerCase(Locale.ROOT) + "%");
}

static Specification<Book> priceAtMost(BigDecimal max) {
    return (book, query, cb) -> cb.lessThanOrEqualTo(book.get("price"), max);
}

static Specification<Book> inCategory(String name) {
    return (book, query, cb) -> {
        Join<Book, Category> categories = book.join("categories");
        return cb.equal(categories.get("name"), name);
    };
}
}

```

Only the filled-in criteria are combined:

```

public Page<BookCard> search(BookFilter filter, Pageable pageable) {
    List<Specification<Book>> criteria = new ArrayList<>();
    if (filter.title() != null) {
        criteria.add(BookSpecifications.titleContains(filter.title()));
    }
    if (filter.maxPrice() != null) {
        criteria.add(BookSpecifications.priceAtMost(filter.maxPrice()));
    }
    if (filter.category() != null) {
        criteria.add(BookSpecifications.inCategory(filter.category()));
    }
    return books.findAll(Specification.allOf(criteria), pageable)
        .map(book -> new BookCard(book.getTitle(), book.getAuthor().getName(),
            ↪ book.getPrice()));
}

```

Expected output ("java" in the title, at most 100, category "java"):

```

== 3.5 Specifications
Effective Java · Joshua Bloch · 89.90
Java Puzzlers · Joshua Bloch · 55.00

```

Its test: catalog/BookSearchTest

i Note

We wrote the field names as strings ("title"). Larger projects prefer the type-safe

metamodel generated by hibernate-processor (Book_.title). It turns a renamed field into a compile error.

3.6 Auditing and Optimistic Locking (@Version)

Goal: keep creation and update timestamps automatically, and prevent lost updates when several users edit at once.

With `@EnableJpaAuditing` (see `JpaAuditingConfiguration`), the `@CreateDate` and `@LastModifiedDate` fields fill themselves. The `@Version` field increases on every update. Hibernate writes `UPDATE ... WHERE id = ? AND version = ?`. If someone changed the row in between, no row matches and an `ObjectOptimisticLockingFailureException` is thrown.

Expected output:

```
== 3.6 Auditing and @Version
price 86.00, version 1, updated 2026-09-23T15:50:27.597813Z
```

Its test: `catalog/AuditingAndVersioningTest`. Two users read the same book. The first one saves, and the second one's stale copy is rejected.

3.7 Pessimistic Locking

Goal: safely decrease a value that invites races, such as stock, with many concurrent buyers.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)           // SELECT ... FOR UPDATE
@Query("select b from Book b where b.isbn = :isbn")
Optional<Book> findForUpdate(String isbn);

@Service
public class StockService {

    private final BookRepository books;

    public StockService(BookRepository books) {
        this.books = books;
    }

    @Transactional
    public boolean trySell(String isbn, int quantity) {
        Book book = books.findForUpdate(isbn).orElseThrow();    // other buyers wait here
        ↪ until we commit
    }
}
```

```

        if (book.getStock() < quantity) {
            return false;
        }
        book.removeStock(quantity); // written at commit
    ↪ (dirty checking)
        return true;
    }
}

```

Expected output (first run on a clean database):

```

== 3.7 Pessimistic locking: 10 buyers, 1 copy each, 3 in stock
stock before 3, sold 3, stock after 0

```

Its test: stock/StockServiceTest. With 10 concurrent buyers and 5 copies, exactly 5 sales happen and the stock ends at zero.

	Optimistic (@Version)	Pessimistic (FOR UPDATE)
How?	The version is compared on update	The row is locked when read
On conflict	Exception. The client retries	Others wait
Fits	Rare conflicts (editing a form)	Frequent conflicts, short operations (stock)

i Note

Without the lock the sales still do not get corrupted, because @Version catches the lost updates. But half of the buyers get an exception. The two mechanisms complement each other.

4. Common Mistakes and Best Practices

!! Caution

spring.jpa.open-in-view is **on** by default and lets lazy relationships load in the web layer, holding a database connection open. It hides N+1 problems and drains the connection pool. In this course we always set it to false.

- **Do:** make every relationship LAZY, and load related data together with @EntityGraph or join fetch when needed.

- **Don't:** return entities directly as JSON. Lazy relationships and infinite loops will hurt you. Use DTOs or projections.
- **Do:** base equals/hashCode on a natural key (ISBN), or carefully on the id. A hashCode based on an id that is null before saving breaks collections.
- **Don't:** update only one side of a bidirectional relationship. Set both sides together with helper methods such as `addLine` (Exercise 1).
- **Do:** use `@Transactional(readOnly = true)` on reading services. Hibernate then skips dirty checking.
- **Don't:** expect batched INSERTs with the `IDENTITY` id strategy. Hibernate cannot batch them. For many rows use a sequence or JDBC batches (Module 05).

5. Summary

- Entities are mutable classes managed by the persistence context. Changes are written at commit.
- Spring Data JPA generates queries, paging and sorting from repository interfaces.
- Measure N+1, and bring it down to one query with `@EntityGraph` or `join fetch`.
- Projections are fast and safe for reading. Specification builds dynamic queries.
- Auditing handles timestamps, and `@Version` handles concurrent updates.
- Use a pessimistic lock (`PESSIMISTIC_WRITE`) for values with frequent conflicts.

Next step: [Exercises](#)

6. Further Reading

- [Spring Data JPA Reference](#)
- [Query Methods & Entity Graphs](#) · [Specifications](#) · [Projections](#)
- [Auditing](#) · [Locking](#)
- [Hibernate ORM 7 User Guide](#)
- [Spring Boot — SQL Databases \(JPA\)](#)